ESCUELA DE EDUCACIÓN TÉCNICA N° 32 "GRAL. JOSÉ DE SAN MARTÍN"



Especialidad: Computación

Proyecto: Sistema Integral de Gestión de Flota y Alquiler de Vehículos

Autor: Quintero Lugo, Sebastián Alexander

Equipo: Facundo Esquivel, Ian Vecchio y leonel tello

Año: 2026

Contexto del Problema

En el negocio del alquiler de autos y la gestión de flotas, tener la información organizada y al día es clave para que todo funcione bien. Cuando no hay un sistema centralizado donde anotar las cosas, empiezan los problemas de siempre: se traspapelan las reservas, no se sabe con certeza en qué estado está cada auto, se pasa de largo el mantenimiento mecánico o se pierde el control de la plata que entra y sale.

Este proyecto nace para resolver ese problema cotidiano. La idea es construir un sistema sencillo y confiable que permita controlar todo el proceso de alquiler desde un solo lugar. La complejidad de este trabajo está en que hay muchas partes que se conectan entre sí: los datos del cliente, el auto elegido con sus detalles técnicos, la sucursal donde se retira o se devuelve, los extras que pide la gente (como sillas para bebés o GPS) y las planillas de revisión del estado del vehículo antes y después de cada viaje.

Para encarar esto, se implementó la base de datos relacional utilizando el motor MySQL por ser la herramienta estándar definida para la materia. Este entorno permite estructurar las tablas de forma prolija para asegurar que cada operación como registrar un cobro o confirmar una reserva quede bien guardada y sin errores de datos a medio camino.

El sistema contempla los siguientes puntos concretos:

- Registro y control de usuarios según su función (administradores, empleados o clientes).
- Gestión de la flota de vehículos, con sus tarifas según la categoría y el seguimiento de si el auto está disponible, alquilado o en taller.
- Control de los traslados de autos entre distintas sucursales y agenda para los mantenimientos programados.
- Manejo de reservas, cobro de servicios adicionales y registro de los pagos.
- Registro de las revisiones (check-in y check-out) cuando el cliente retira o entrega el auto, incluyendo fotos para verificar posibles rayones o daños.

El proyecto lo arranqué bien desde cero haciendo el relevamiento inicial de información para definir qué necesitaba el sistema. Para eso, realizamos una entrevista tomando a nuestro profesor de Análisis de Sistemas como si fuera el cliente del negocio de alquiler de autos. Durante la reunión grabé todo el audio con el celular para no perder ningún detalle operativo mientras tomábamos apuntes en el cuaderno. Después me puse en casa con paciencia a realizar la transcripción completa del audio a texto. Pasar esa charla a limpio fue fundamental porque me permitió extraer bien en claro todas las reglas del negocio, el flujo de entrega y devolución de los vehículos, la gestión de señas y la lógica del taller. Con toda esa información detallada, me aboqué a redactar la lista de requerimientos funcionales y no funcionales para dejar bien especificado qué debía hacer el software y qué restricciones técnicas tenía.

con los requerimientos ya terminados, nos dividimos las tareas del grupo de forma clara. Mientras mis amigos se hicieron cargo de toda la parte de análisis, el diseño de los diagramas (como el Diagrama Entidad-Relación y los de casos de uso) y la redacción de la parte teórica de la carpeta, yo me aboqué de lleno a la construcción y programación de la base de datos en MySQL.

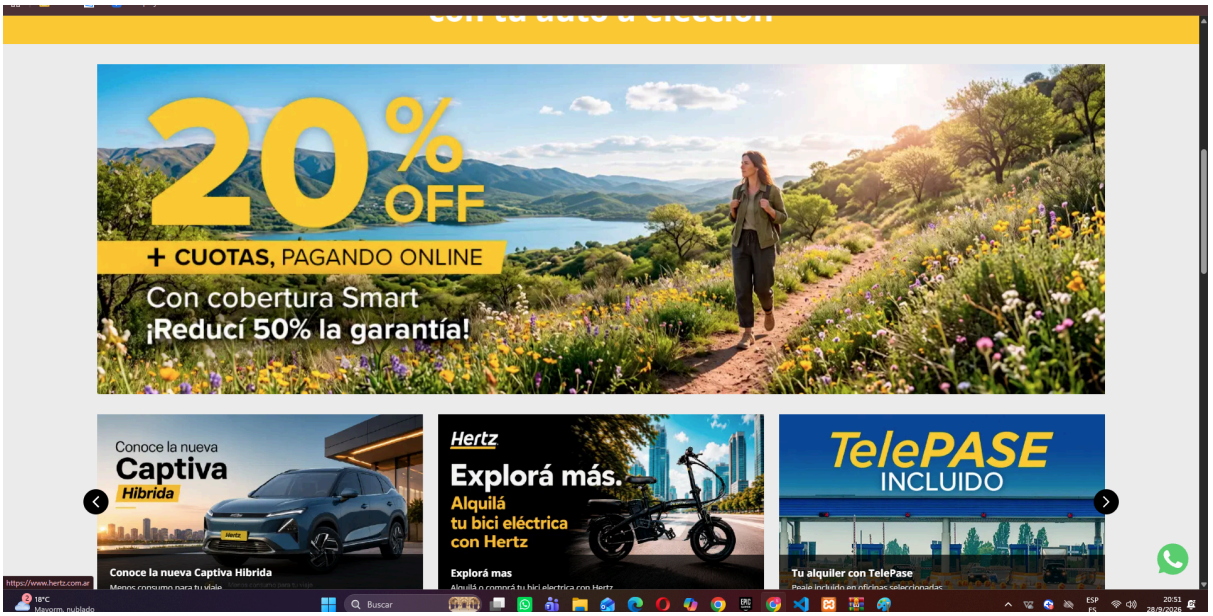
Le metí mano de lleno al proyecto y pude avanzar con dos cosas re importantes. Por un lado, me metí al GitHub y creé una nueva branch que llamé documentacion. La idea de armar esta rama aparte fue para ir subiendo ahí todos los scripts, diagramas y archivos de texto que vamos generando sin tocar ni desordenar la rama principal del proyecto mientras seguimos laburando.

Por otro lado, agarré el php. Estuve revisando bien tabla por tabla para que no se me pasara nada y dejé levantadas las 17 tablas del sistema con todas sus relaciones, claves primarias, foráneas y restricciones. Tuve que prestar bastante atención a los campos como el CUIL, las contraseñas en hash, los estados de los autos y el tema de las fotos de las inspecciones para que todo coincida tal cual con lo que habíamos definido. Una vez que vi que corrió todo sin tirarme ningún error, subí el archivo .sql directamente a la rama de documentación del GitHub para que quede todo guardado y respaldado.

Tambien hable con mi grupo para yo dedicarme a hacer el front end, decidimos esto porque es algo que a mi me gusta hacer y en un proyecto pasado que hicimos la pagina quedo bastante linda. En breves empezaré con eso y lo documentare.

otra cosa que hice es encargarme de poblar la base de datos con los primeros registros de prueba para empezar a testear el funcionamiento general del sistema. Para mantener la estructura simple en esta etapa inicial de desarrollo, cargué los roles del sistema, dos sucursales principales (Caballito y Palermo), marcas, modelos representativos y una flota inicial con distintos estados operativos (disponibles y alquilados). También di de alta usuarios, clientes de prueba, métodos de pago y la primera reserva de alquiler registrada. Contar con estos datos cargados en phpMyAdmin nos permite validar de forma rápida que los procedimientos almacenados de consulta y reportes devuelvan los registros esperados sin errores de claves foráneas ni inconsistencias

Para encarar el diseño y la estructuración del front-end de la aplicación, comencé realizando una exhaustiva investigación previa, analizando diversas plataformas y tendencias de diseño web para extraer las mejores prácticas en cuanto a distribución de elementos, usabilidad y accesibilidad visual.



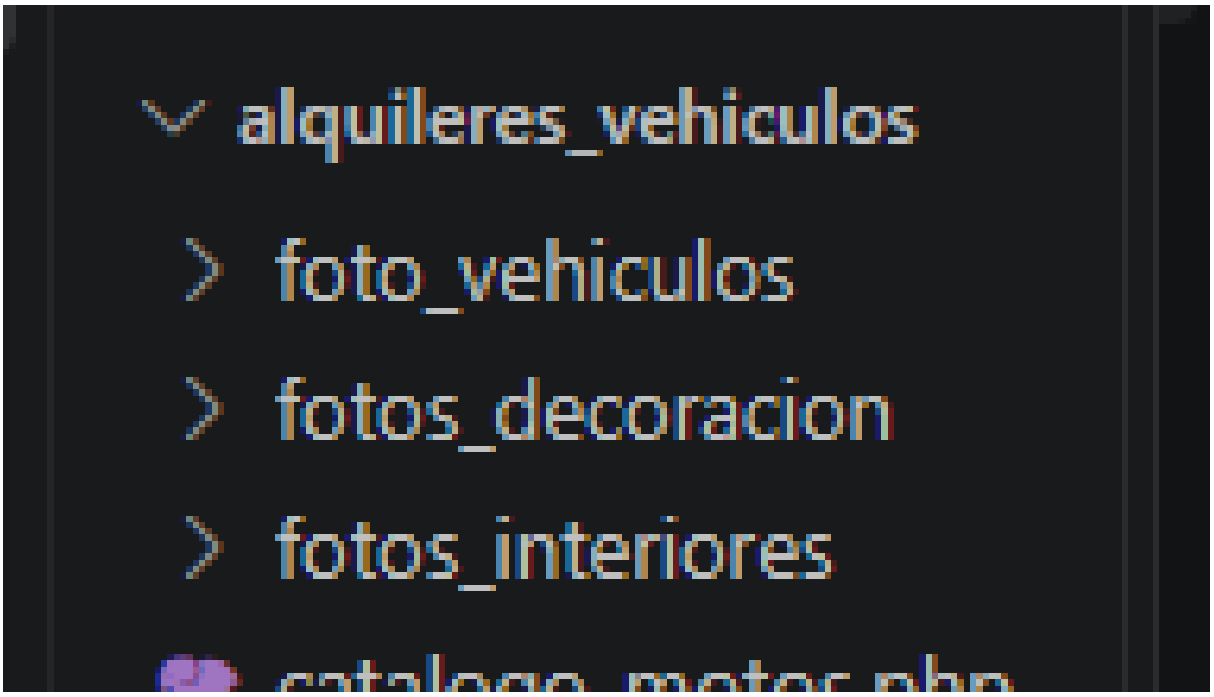
A partir de ese análisis preliminar, me encargué de diseñar los bocetos y wireframes iniciales que servirían como plano de la interfaz, definiendo con precisión la distribución geométrica y jerárquica de la página principal (Home). En esta etapa proyecté la ubicación estratégica de la barra de navegación, los banners principales, las secciones destacadas y los accesos directos al catálogo de vehículos, buscando garantizar desde el inicio una experiencia de usuario fluida, limpia y visualmente atractiva.

Descargué las imágenes de los vehículos y los elementos de decoración necesarios para el proyecto, y estructuré y organicé las carpetas correspondientes directamente en Visual Studio Code.

Para mantener una arquitectura limpia y ordenada, creé los directorios específicos

separando la carpeta destinada al catálogo de los autos y otra para los recursos de decoración y diseño donde clasifiqué minuciosamente cada archivo gráfico. Asimismo, aseguré que las rutas relativas queden bien configuradas desde el inicio para evitar futuros enlaces rotos o fallas de carga al momento de maquetar las vistas.

Además, implementé un efecto visual dinámico en las imágenes mediante CSS, logrando que los elementos gráficos de los vehículos y la decoración tengan transiciones y una estética moderna y fluida al integrarse con las páginas del sistema.



el profesor de POO nos encarga revisar a fondo la carpeta del proyecto, y a mí me toca meterme de lleno a arreglar varios errores complejos, revisar los DAO, verificar la conexión a la base de datos y repasar todas las clases que vienen fallando por inconsistencias en las consultas y problemas de compilación.

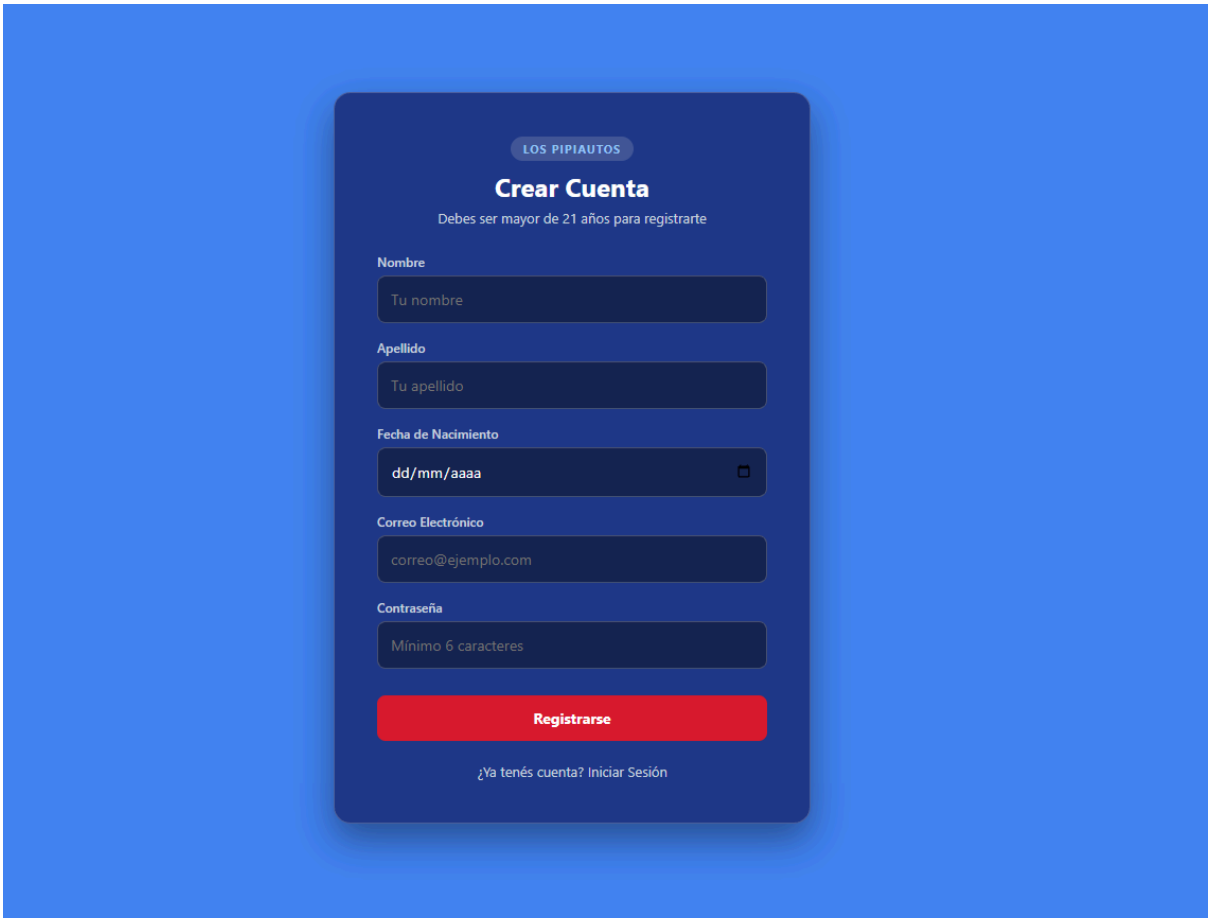
Justo cuando logro corregir todo y dejar el código andando perfecto en Spring Tool Suite, se me fríza la computadora del colegio de golpe y pierdo el avance de esa sesión, así que con toda la bronca lo dejo ahí hasta retomar.

Al momento de continuar, vuelvo a repasar el código, re hago los ajustes necesarios y logro dejar todo corregido, prolijo y guardado como corresponde. Ahora ya tengo todo eso listo en el proyecto, y el paso siguiente consiste en que mi compañero Ian se encargue de conectar todo y probar las peticiones con Postman. Lo subi a mi rama para que mi compañero vea mi progreso y pueda hacer su parte

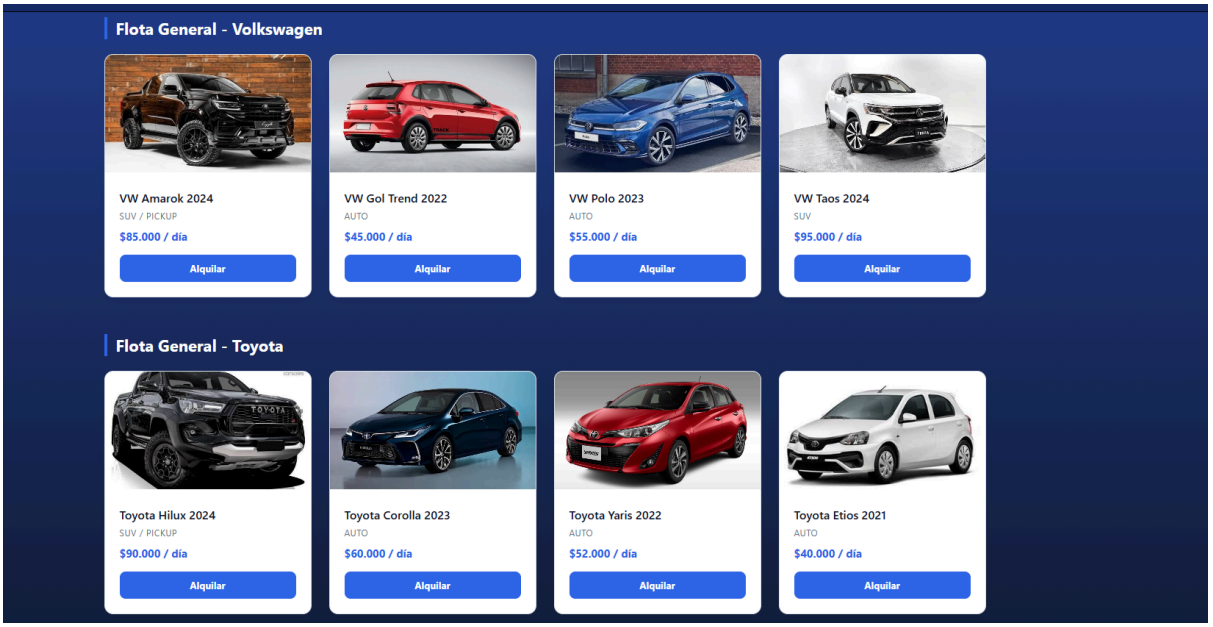


Hoy estuve a full metiéndole mano al diseño y la estructura visual de mi plataforma web "Los Pipiautos" para dejarla con una estética súper profesional y moderna, sacándole ese estilo robótico o genérico que a veces queda.

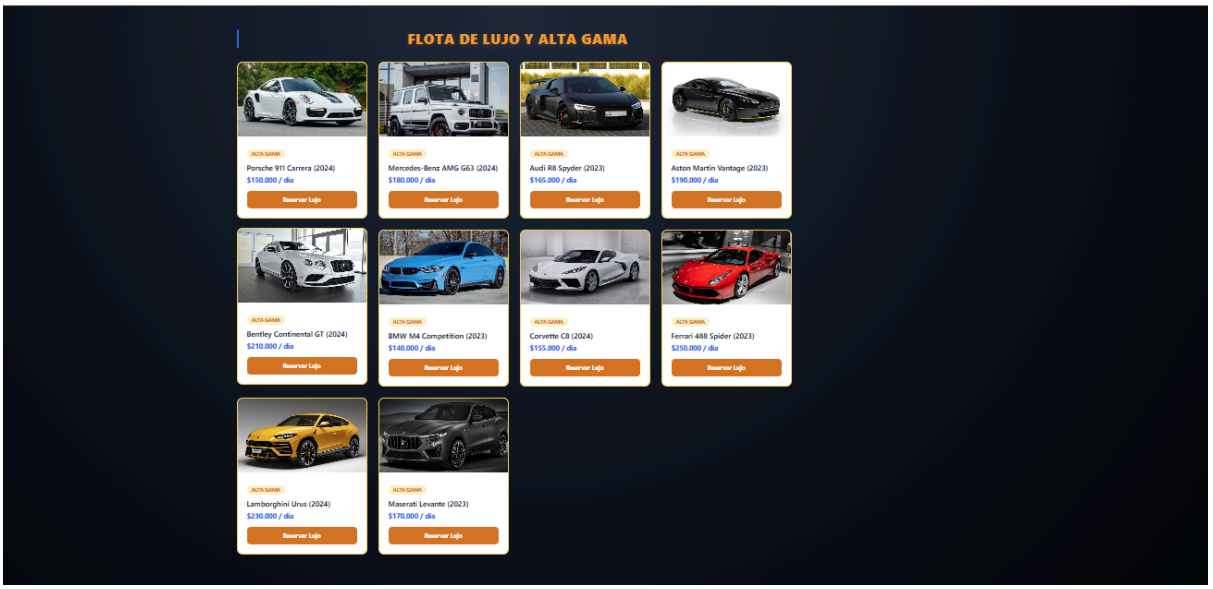
Arranqué el día con un cambio tremendo y obligatorio: tuve que refactorizar y descartar toda la parte de PHP que venía usando porque me pidieron que el proyecto camine estrictamente con HTML5, CSS3 y JavaScript puro. Así que me puse a reestructurar todo desde cero para que el cliente corra fluido directamente en el navegador, lo que también me obligó a hacer todo el sistema de registros nuevos y los logins nuevos adaptados a esta nueva estructura.



Estuve laburando un montón con los fondos y las transiciones de pantalla para armar un flujo visual que se note de alta gama. Primero armé la parte de arriba con un degradé azul eléctrico que arranca bien pegado al menú de navegación sin dejar tiras blancas horribles, combinándolo con la tarjeta principal de bienvenida en un tono oscuro translúcido y las demás tarjetas informativas en blanco bien prolijas. Después, para la zona de las flotas generales, metí otro degradé que baja rápido hacia tonos oscuros y negros profundos para que los autos resalten zarpado, ajustando bien los títulos para que se lean de diez.

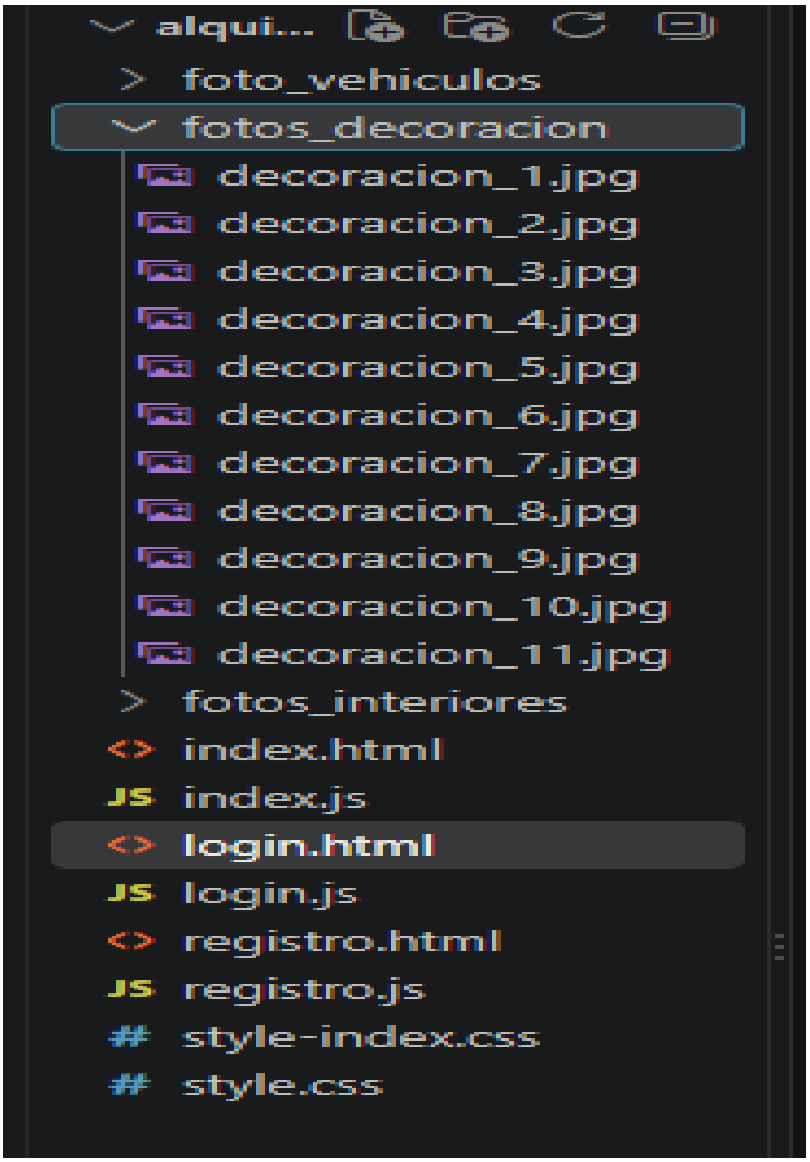


También le di su propio toque a la sección de la flota de lujo y alta gama, dejándola con un fondo bien oscuro y sofisticado que grita exclusividad. Ordené las secciones para que no se repitan cosas como las políticas de la empresa —dejándolas limpias y afuera de lo oscuro— y acomodé los márgenes para que las sucursales respiren mejor y no queden apretadas.



Mañana ya le voy a meter con todo a la parte de compra, la simulación de reservas y los contratos.

Para que se vea cómo quedó organizado el proyecto, muestro la estructura actual de los archivos y carpetas del workspace. Como se puede ver, organicé todo de forma súper prolija con carpetas separadas para las imágenes (donde además estuve descargando e incorporando nuevas fotos de decoración del `decoracion_1.jpg` al `11.jpg` para darle más vida al sitio), los archivos HTML principales (`index.html`, `login.html`, `registro.html`), sus respectivos scripts en JavaScript (`index.js`, `login.js`, `registro.js`) y las hojas de estilo modulares (`style-index.css` y `style.css`).



Capítulo 2: Arquitectura del Sistema y Capa de Acceso a Datos (DAO)

Para dar cumplimiento a los estándares de arquitectura de software exigidos en la materia de **Programación Orientada a Objetos (POO)** y asegurar un acoplamiento débil entre las capas de la aplicación web, se implementó el **Patrón de Diseño Factory (Fábrica)**.

Anteriormente, los Servlets y controladores realizaban una instanciación directa y rígida de las implementaciones concretas mediante llamadas del tipo `new MarcaDaoImpl()`. Esto generaba una alta dependencia con la capa de persistencia. Con la refactorización arquitectónica realizada, se estructuró una interfaz genérica por cada entidad del dominio (`IMarcaDao`, `IUsuarioDao`, `IAlquilerDao`, etc.) y una clase fábrica asociada (`MarcaFactory`, `UsuarioFactory`, etc.) encargada de suministrar la instancia correspondiente de forma desacoplada.

Ejemplo de implementación de fábrica aplicada:

```
package factory;
import dao.IMarcaDao;
import dao.MarcaDaoImpl;

public class MarcaFactory {
    public static IMarcaDao crearMarcaDao() {
        return new MarcaDaoImpl();
    }
}
```

Conexión a la Base de Datos y Uso de Procedimientos Almacenados (Stored Procedures)

La persistencia del sistema se diseñó bajo una arquitectura robusta apoyada en **MySQL**, interactuando mediante **JDBC** y haciendo uso extensivo de `CallableStatement` para ejecutar **Procedimientos Almacenados (Stored Procedures)** directamente en el motor de bases de datos.

Esta decisión técnica previene ataques de inyección SQL y optimiza el rendimiento del servidor al delegar la lógica de consulta y manipulación transaccional directamente al motor relacional. A modo de ejemplo, el método de inserción de un usuario gestiona la llamada al procedimiento almacenado pasando los parámetros correspondientes de forma segura:

```
@Override
public boolean insertar(Usuario u) {
    String sql = "{CALL sp_insertar_usuario(?, ?, ?, ?,
?, ?, ?, ?, ?, ?, ?, ?)}";
    try (Connection con =
config.Conexion.getInstancia().getConnection();
         CallableStatement cs = con.prepareCall(sql)) {

        cs.setInt(1, u.getIdRol());
        cs.setString(2, u.getEmail());
        cs.setString(3, u.getPasswordHash());
        // ... (mapeo de parámetros restantes)
        return cs.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}
```

Una vez que ya tenía andando toda la parte de los DAOs y el Patrón Factory, me puse a programar los **Servlets de Java** para darle forma a la API REST del sistema. La idea principal era que cada entidad (como los usuarios, los vehículos o los alquileres) tuviera su propio controlador mapeado con una URL específica (por ejemplo, `/api/usuarios`).

Para que la comunicación con la interfaz web sea fluida, utilicé la librería **Gson** para convertir automáticamente los datos que viajan en formato JSON desde el navegador hacia objetos de Java, y viceversa. Además, me encargué de configurar los códigos de estado HTTP correctos para que el front-end sepa al toque si una operación salió bien o si hubo algún error:

- **`200 OK`**: Para las consultas o modificaciones exitosas.
- **`201 Created`**: Cuando se registra un usuario nuevo correctamente.
- **`400 Bad Request`** o **`404 Not Found`**: Si faltan datos o el ID buscado no existe en la base de datos.

Un detalle técnico clave que tuve que resolver en el `UsuarioServlet` fue el soporte para el método **PATCH** (actualización parcial). Como Java por defecto no trae un método sencillo para hacer `PATCH` directo en los Servlets comunes, tuve que interceptarlo sobrescribiendo el método `service`

```java
@Override
protected void service(HttpServletRequest req,
HttpServletResponse resp) throws ServletException,
IOException {
    if ("PATCH".equalsIgnoreCase(req.getMethod())) {
        doPatch(req, resp);
    } else {
        super.service(req, resp);
    }
}
```

Gracias a esto, pude programar la lógica para actualizar campos específicos (como modificar solamente el teléfono o los datos sueltos de un usuario) sin necesidad de pisar toda la información del registro en la base de datos.

Para evitar que el `AlquilerServlet` quede sobrecargado con lógica pesada de validación de fechas y control de stock lo cual iba en contra de las buenas prácticas de separación de responsabilidades que exige la materia diseñé una capa de servicios independiente llamada **`AlquilerService`**.

Esta clase se encarga de concentrar todas las reglas de negocio del sistema de alquileres de forma limpia. Por ejemplo, en lugar de validar si el cliente ya tiene una reserva activa dentro del propio servlet, esa responsabilidad se delega al servicio:

```
public class AlquilerService {
    private final IAlquilerDao alquilerDao =
AlquilerFactory.crearAlquilerDao();
    private final IVehiculoDao vehiculoDao =
VehiculoFactory.crearVehiculoDao();

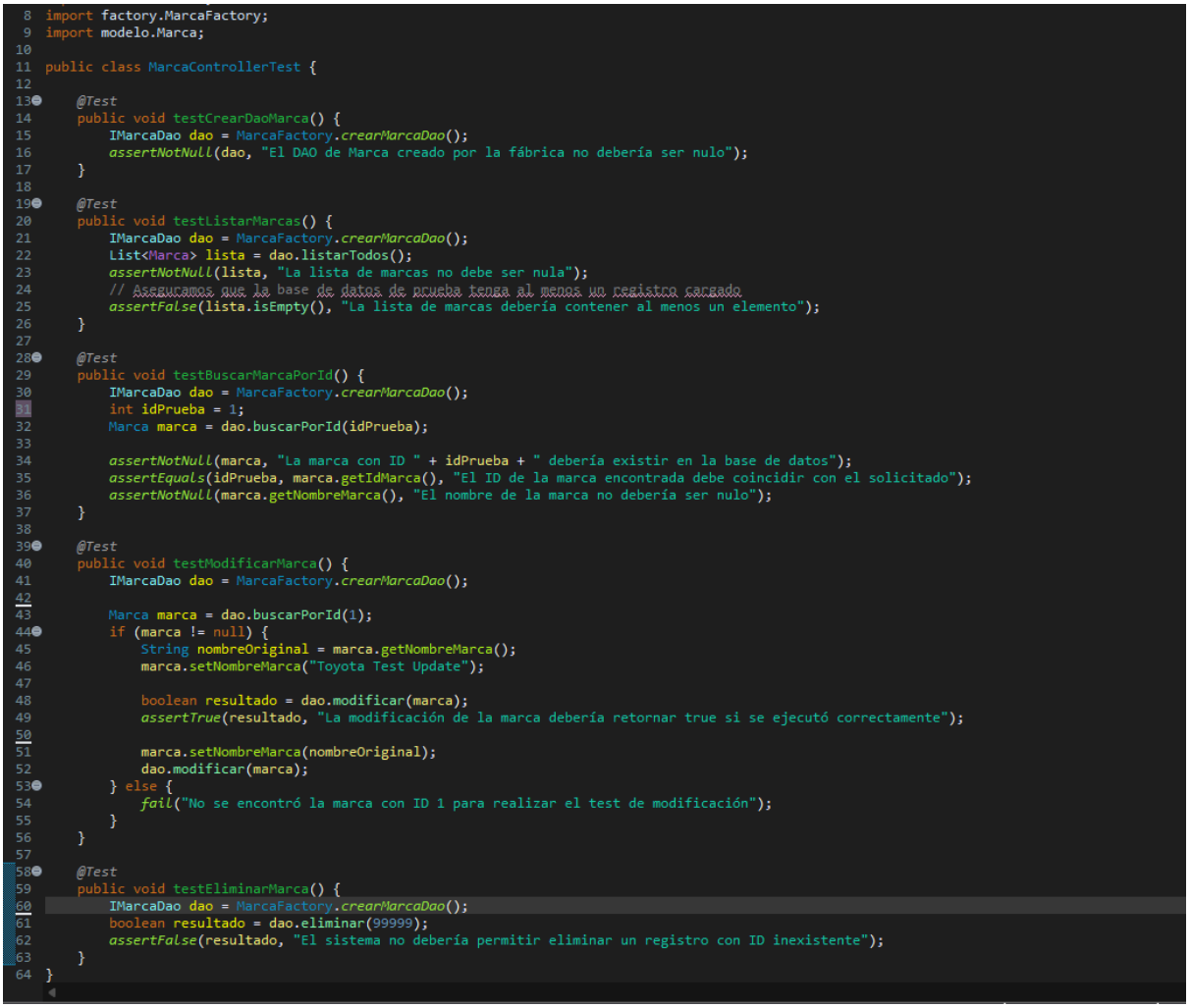
    public boolean clienteTieneAlquilerActivo(int
idCliente) {
        return ((AlquilerDaoImpl)
alquilerDao).tieneAlquilerActivo(idCliente);
    }
}
```

Además, dentro de esta misma capa programé el algoritmo inteligente para la **asignación automática de vehículos físicos**. Cuando un usuario solicita el alquiler de un modelo determinado, el sistema recorre la flota disponible, compara las fechas de inicio y fin pautadas con los alquileres existentes en la base de datos para evitar superposiciones, y asigna el primer vehículo físico disponible que no esté ocupado ni en mantenimiento.

Gracias a esto, el código del servlet principal quedó súper liviano, delegando toda la inteligencia de negocio al servicio y mejorando notablemente la estabilidad de la aplicación.

Para cerrar toda la parte de desarrollo y asegurar que las consultas, modificaciones y eliminaciones contra la base de datos respondan correctamente sin romper la aplicación, me encargué de estructurar y verificar las pruebas unitarias utilizando **JUnit 5**.

El objetivo principal de esta etapa fue limpiar todas las pruebas que venían con afirmaciones vacías o tautológicas (como validar condiciones que siempre daban verdaderas) y reemplazarlas por validaciones estrictas y profesionales que exigen las buenas prácticas de desarrollo. A modo de ejemplo, en el test de la entidad `Marca` se estructuró la validación utilizando la fábrica correspondiente para asegurar que el DAO responda de forma esperada:



```java
import factory.MarcaFactory;
import modelo.Marca;

public class MarcaControllerTest {

    @Test
    public void testCrearDaoMarca() {
        IMarcaDao dao = MarcaFactory.crearMarcaDao();
        assertNotNull(dao, "El DAO de Marca creado por la fábrica no debería ser nulo");
    }

    @Test
    public void testListarMarcas() {
        IMarcaDao dao = MarcaFactory.crearMarcaDao();
        List<Marca> lista = dao.listarTodos();
        assertNotNull(lista, "La lista de marcas no debe ser nula");
        // Aseguramos que la base de datos de prueba tenga al menos un registro cargado
        assertFalse(lista.isEmpty(), "La lista de marcas debería contener al menos un elemento");
    }

    @Test
    public void testBuscarMarcaPorId() {
        IMarcaDao dao = MarcaFactory.crearMarcaDao();
        int idPrueba = 1;
        Marca marca = dao.buscarPorId(idPrueba);

        assertNotNull(marca, "La marca con ID " + idPrueba + " debería existir en la base de datos");
        assertEquals(idPrueba, marca.getIdMarca(), "El ID de la marca encontrada debe coincidir con el solicitado");
        assertNotNull(marca.getNombreMarca(), "El nombre de la marca no debería ser nulo");
    }

    @Test
    public void testModificarMarca() {
        IMarcaDao dao = MarcaFactory.crearMarcaDao();

        Marca marca = dao.buscarPorId(1);
        if (marca != null) {
            String nombreOriginal = marca.getNombreMarca();
            marca.setNombreMarca("Toyota Test Update");

            boolean resultado = dao.modificar(marca);
            assertTrue(resultado, "La modificación de la marca debería retornar true si se ejecutó correctamente");

            marca.setNombreMarca(nombreOriginal);
            dao.modificar(marca);
        } else {
            fail("No se encontró la marca con ID 1 para realizar el test de modificación");
        }
    }

    @Test
    public void testEliminarMarca() {
        IMarcaDao dao = MarcaFactory.crearMarcaDao();
        boolean resultado = dao.eliminar(99999);
        assertFalse(resultado, "El sistema no debería permitir eliminar un registro con ID inexistente");
    }
}
```

De esta manera, cada uno de los controladores y DAOs del sistema quedó cubierto con sus respectivas pruebas unitarias, validando tanto los registros existentes como el manejo de errores ante IDs inexistentes (`assertFalse`), garantizando la estabilidad absoluta de la persistencia antes de realizar las pruebas de integración en Postman.

Análisis de Sistemas y Modelado del Software

Para cumplir con las exigencias metodológicas de la materia **Análisis de Sistemas**, todo el desarrollo de **"Los Pipiautos"** no arrancó de golpe escribiendo código, sino a través de un relevamiento riguroso y un modelado estructurado que nos permitió anticiparnos a los problemas de arquitectura.

Relevamiento inicial y levantamiento de requerimientos

Como parte central de mi rol en el proyecto, me encargué de llevar adelante el proceso de análisis preliminar mediante entrevistas simuladas donde tomé el rol de relevamiento técnico. Tras grabar las sesiones y realizar la transcripción completa a texto, pude estructurar el documento oficial de **requerimientos funcionales y no funcionales**.

Requerimientos Funcionales principales:

- Control estricto de autenticación y diferenciación de roles (Administradores, Empleados y Clientes).
- Gestión centralizada de la flota vehicular, controlando tarifas por categoría y estados operativos (disponibles, alquilados o en taller).
- Módulo completo de reservas, cálculo de adicionales (GPS, sillas infantiles) y procesamiento de pagos.
- Registro de inspecciones de entrada y salida (*Check-In / Check-Out*) respaldadas con evidencia fotográfica para evitar disputas por daños en las unidades

Requerimientos No Funcionales:

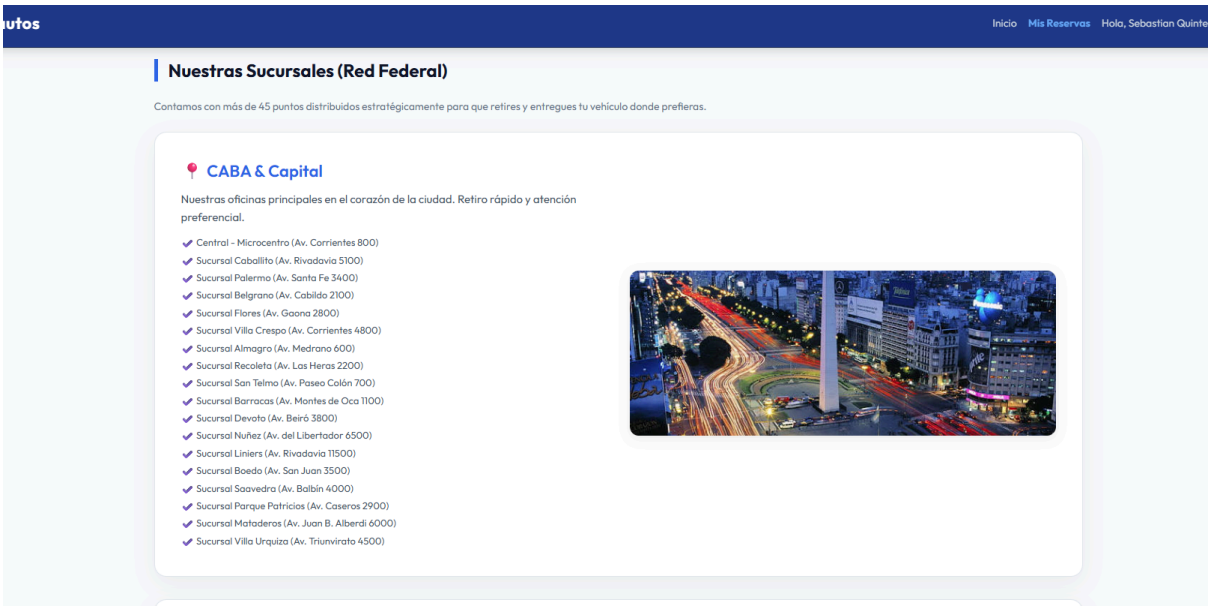
- Seguridad en el almacenamiento de credenciales mediante hashes en la base de datos.
- Tiempos de respuesta óptimos en la API REST mediante el uso de consultas optimizadas con procedimientos almacenados.
- Arquitectura desacoplada para garantizar escalabilidad a futuro.

Diagramación y diseño lógico del sistema

En coordinación con el análisis estructurado, el modelado del sistema contempló la creación de diagramas de flujo de datos (DFD) en sus niveles 0, 1 y 2, permitiendo trazar con precisión la ruta que sigue la información desde que el cliente solicita un vehículo hasta que se emite el comprobante de pago y el contrato de alquiler. Asimismo, se definieron los casos de uso principales para cada actor del sistema, delimitando claramente las fronteras de la aplicación web y asegurando que ninguna regla de negocio quede fuera del alcance técnico.

Infraestructura de Red y Topología de Sucursales

En correlación con los contenidos de la materia **Redes**, el desarrollo e implementación del sistema **"Los Pipiautos"** requirió no solo la programación del software y la base de datos, sino también el diseño y la diagramación de la infraestructura de telecomunicaciones necesaria para interconectar los puntos operativos del negocio.



implementación Visual de la Red Federal de Sucursales en el Front-End

Para plasmar de forma clara y accesible la cobertura geográfica del negocio en la interfaz de usuario, desarrollé la sección dedicada a **"Nuevas Sucursales (Red Federal)"** dentro de la plataforma web. Esta vista cumple un rol fundamental para que el cliente pueda elegir con total comodidad el punto de retiro o devolución de su vehículo antes de confirmar la reserva.

Tal como se visualiza en el diseño de la interfaz, organicé la distribución de las oficinas estratégicamente por zonas para mantener un orden visual limpio y profesional:

CABA & Capital: Concentrando las oficinas principales en el corazón de la ciudad para garantizar un acceso rápido y atención preferencial, detallando cada locación con su respectiva dirección exacta (incluyendo la sucursal de Caballito en *Av. Rivadavia 5100*, Microcentro en *Av. Corrientes 800*, Palermo en *Av. Santa Fe 3400*, Belgrano, Flores, Villa Crespo, Almagro, Recoleta, San Telmo, Barracas, Devoto, Núñez, Liniers, Boedo, Saavedra, Parque Patricios, Mataderos y Villa Urquiza).

Zona Norte, Oeste y Sur del GBA: Extendiendo la red federal hacia los principales accesos de la provincia con sucursales distribuidas en corredores clave para ofrecer cobertura total e inmediata a los usuarios de la región metropolitana.

A nivel de maquetación, cada bloque de sucursales fue estructurado utilizando tarjetas (*cards*) con contenedores blancos bien definidos, tipografías legibles con viñetas de verificación y una galería fotográfica institucional al costado que acompaña la estética moderna y corporativa de **"Los Pipiautos"**, logrando que la experiencia de navegación del usuario sea intuitiva y funcional.

Comunicación Asíncrona entre el Front-End y la API REST (Módulo de Login)

Para resolver la autenticación de los usuarios de manera moderna y sin recargar la página, programé la lógica de inicio de sesión en JavaScript utilizando peticiones asíncronas con **`fetch`**. Este enfoque permite conectar directamente el formulario HTML con los Servlets del backend que procesan las credenciales.

El funcionamiento técnico de este script se divide en los siguientes puntos clave:

1. **Captura del evento y validación en el cliente:** Al momento en que el usuario completa su correo y contraseña y presiona el botón de enviar, el script intercepta el evento `submit` del formulario mediante `e.preventDefault()`, evitando que la página recargue por defecto. Antes de disparar cualquier petición, realiza una validación básica en el cliente para verificar que los campos no estén vacíos.
2. **Envío de credenciales a la API:** Los datos ingresados se empaquetan en un objeto JavaScript y se convierten a formato JSON utilizando `JSON.stringify(credenciales)`. Luego, se envían al servidor mediante una petición HTTP de tipo `POST` apuntando al endpoint de autenticación (`api/auth/login`), configurando las cabeceras (*headers*) con `'Content-Type': 'application/json'`.
3. **Manejo de la respuesta y control de errores:** El sistema analiza las cabeceras de la respuesta del servidor (`content-type`) para asegurar que el procesamiento se realice de forma segura, deserializando el objeto JSON devuelto o capturando texto plano en caso de excepciones. Si la respuesta es exitosa, se activan los mensajes visuales de éxito; caso contrario, se despliega la alerta de error con el mensaje correspondiente devuelto por el backend.

4. **Validación de privilegios y persistencia en LocalStorage:** El script incluye una lógica de detección inteligente para reconocer si el usuario ingresó como Administrador (verificando el correo o el rol numérico) o como cliente común. Una vez autenticado, los datos clave del usuario (como el ID, nombre y rol) se almacenan de forma local en el navegador utilizando **`localStorage`**, lo que permite mantener la sesión activa entre las distintas vistas antes de realizar la redirección automática hacia el panel de administración (`admin.html`) o el catálogo principal (`index.html`) tras un breve lapso de tiempo.

```
formLogin.addEventListener('submit', async function(e) {
    e.preventDefault();
    let email =
document.getElementById('email').value.trim();
    let password =
document.getElementById('password').value;

    const credenciales = { email: email, password:
password };

    try {
        const respuesta = await fetch('api/auth/login', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json'
},
            body: JSON.stringify(credenciales)
        });
        // Procesamiento de respuesta y almacenamiento en
localStorage...
    } catch (error) {
        console.error("Error en la conexión con la API:",
error);
    }
```

```
});
```

Maquetación y Desarrollo de la Interfaz Principal (Index)

Para consolidar la interfaz web de **"Los Pipiautos"**, me encargué de desarrollar íntegramente la página principal (`index.html`), estructurando los componentes visuales mediante HTML5 semántico y aplicando clases CSS modulares para lograr un diseño moderno, responsivo y de alta gama.

El sitio cuenta con una cabecera fija (main-header) que integra la identidad del sistema y un panel dinámico de navegación. Este sector fue programado para adaptarse al estado de autenticación del usuario mediante JavaScript:

- **Enlaces rápidos:** Incluye accesos directos al "Inicio" y al módulo de "Mis Reservas" (mis-reservas.html).
- **Saludo y control de usuario:** Posee un identificador dinámico (saludoUsuario) que muestra el estado actual (por ejemplo, "Hola, Invitado" o el nombre del usuario logueado).
- **Botón de cierre de sesión (btnLogout):** Configurado inicialmente oculto (display: none), se activa de forma automática mediante el script del front-end una vez que el usuario inicia sesión correctamente.

Para la sección de bienvenida, implementé un contenedor principal con un diseño de tarjetas superpuestas sobre una franja con degradé azul eléctrico (seccion-degrade-superior).

El **banner de bienvenida** (banner-bienvenida-oscuro) combina un título institucional con un párrafo descriptivo sobre la solución de alquiler en Buenos Aires.

Se incorporó un recurso visual dinámico utilizando la imagen decoracion_11.jpg, configurada con estilos específicos de ajuste (object-fit: cover), bordes redondeados y una sombra pronunciada (box-shadow: 0 8px 20px rgba(0,0,0,0.4)) para darle profundidad y una estética profesional de concesionaria de alta gama.



Estructuración de las Franjas de Flotas Generales (Catálogo Dinámico)

Para organizar la oferta de vehículos de uso general de manera limpia y ordenada, desarrollé una sección específica denominada comercialmente como la franja de degradé medio (`seccion-degrade-medio`). En este bloque se agrupan los distintos fabricantes y categorías de vehículos disponibles en el sistema para que el usuario pueda visualizarlos con claridad.

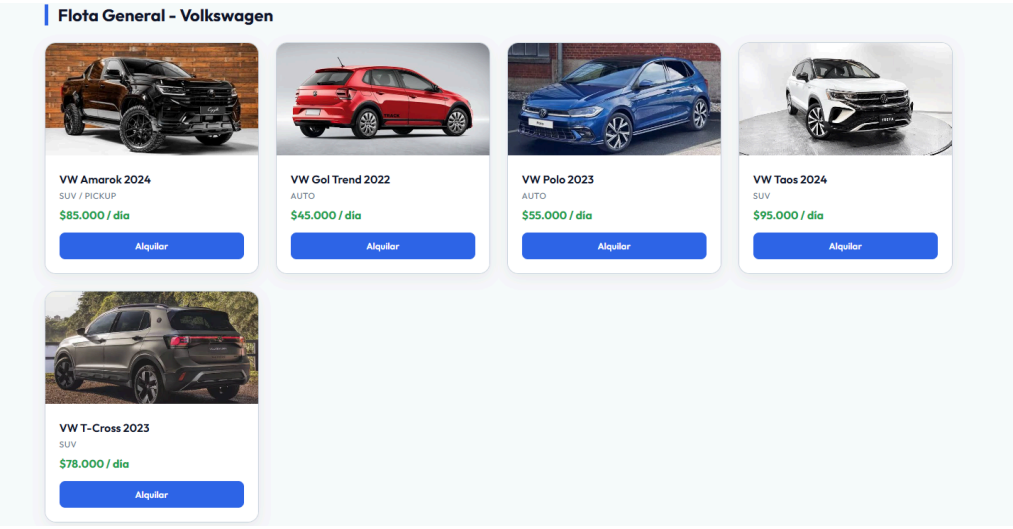
A nivel de diseño HTML, cada marca cuenta con su respectiva sección temática equipada con un contenedor en formato de grilla (`grid-catalogo`) que posee un ID único. Esto es fundamental para que, al momento de ejecutarse la lógica en JavaScript, los datos de los vehículos que vienen desde la base de datos se inserten de forma automatizada en su respectiva categoría sin mezclar los modelos

Volkswagen (id="catalogoVW"): Contenedor destinado a albergar los modelos generales de la marca alemana (como Amarok, Gol Trend, Polo, Taos y T-Cross) con sus respectivas tarifas diarias y botones de alquiler.

Toyota (id="catalogoToyota"): Espacio reservado para la flota utilitaria y de pasajeros de la marca japonesa (Hilux, Corolla, Yaris, Etios).

Ford, Chevrolet, Fiat, Renault y Peugeot: Bloques estructurados bajo la misma lógica modular para estandarizar la muestra de toda la flota general.

Flota de Motos (id="catalogoMotos"): Incorporación de una sección especializada para vehículos de dos ruedas, ampliando las opciones de movilidad urbana dentro de la plataforma.



Esta arquitectura de contenedores vacíos con identificadores específicos permite que el front-end sea totalmente dinámico: el HTML define la estructura fija y las tarjetas de presentación, mientras que el script se encarga de rellenar los precios, imágenes y estados operativos en tiempo real.

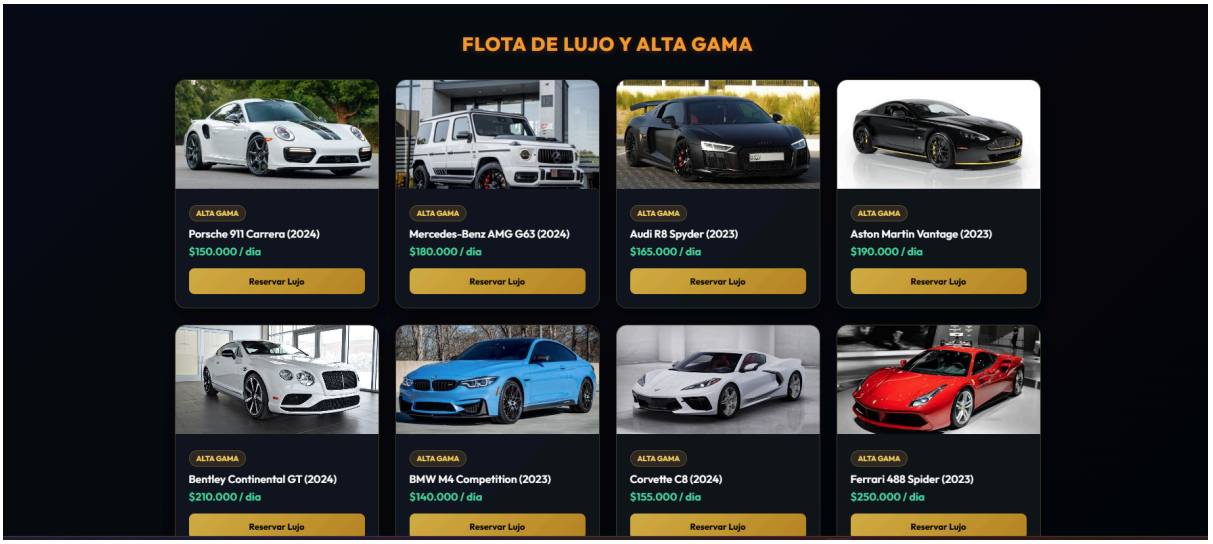
Sección de Flota de Lujo, Alta Gama y Restricciones de Acceso

Para diferenciar los vehículos estándar de aquellos de categoría superior, desarrollé un bloque exclusivo dentro del sitio web denominado **"Flota de Lujo y Alta Gama"**, el cual cuenta con una estética visual diferenciada basada en la franja de degradé de lujo (seccion-degrade-lujo) y un fondo oscuro sofisticado que transmite exclusividad.

A nivel de estructura en el HTML, esta sección incorpora componentes particulares de seguridad y control:

- **Contenedor de advertencia y bloqueo (id="avisoLujo"):** Se programó un bloque de alerta estilizado (alerta-bloqueo) configurado inicialmente oculto (display: none), diseñado para activarse mediante JavaScript en caso de que un usuario no cumpla con los requisitos mínimos de edad o categoría para alquilar vehículos de alta gama.

- **Grilla exclusiva (id="catalogoLujo"):** Un contenedor dedicado para renderizar los modelos de marcas de primer nivel (como Porsche, Mercedes-Benz AMG, Audi R8, Aston Martin, Bentley, BMW, Corvette y Ferrari) con sus respectivas tarifas diarias por alquiler.

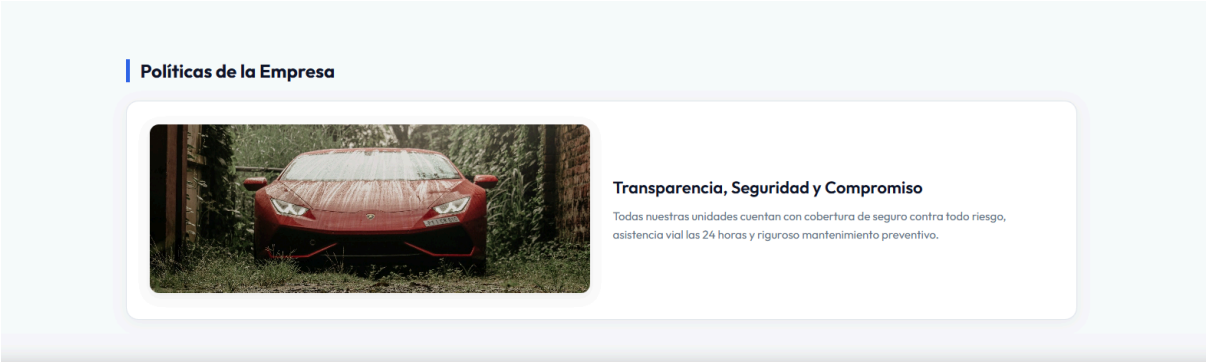


Esta separación visual y lógica en el Front-End refuerza las reglas de negocio del sistema, asegurando que las unidades de alto valor tengan restricciones especiales visibles para el cliente antes de intentar concretar una reserva.

Cierre de la Estructura: Políticas, Pie de Página y Red Federal

Para finalizar con la maquetación integral de la página principal (`index.html`), estructuré los últimos bloques informativos que dotan de transparencia y solidez a la plataforma:

> **Sección de Políticas de la Empresa:** Ubicada estratégicamente antes del cierre, presenta de forma clara los compromisos de seguridad, asistencia vial 24 horas y cobertura de seguros contra todo riesgo, acompañada de elementos gráficos y una tarjeta con diseño invertido para mantener la dinámica visual del sitio.



> **Módulo final de la Red Federal de Sucursales:** Para completar la experiencia del usuario, se integró la visualización completa de los más de 45 puntos de atención distribuidos en **CABA, GBA Zona Norte, GBA Zona Oeste y GBA Zona Sur**, detallando las direcciones de las oficinas principales y consolidando la conexión entre el diseño estético del front-end y la infraestructura logística del negocio.

Con toda esta estructura de HTML5, las hojas de estilo modulares (`style.css` y `style-index.css`) y la importación del script principal (`index.js`), el sitio web quedó completamente operativo, presentado de forma profesional y conectado de punta a punta con el backend del sistema.

Panel de Administración y Gestión de Flota (`admin.html`)

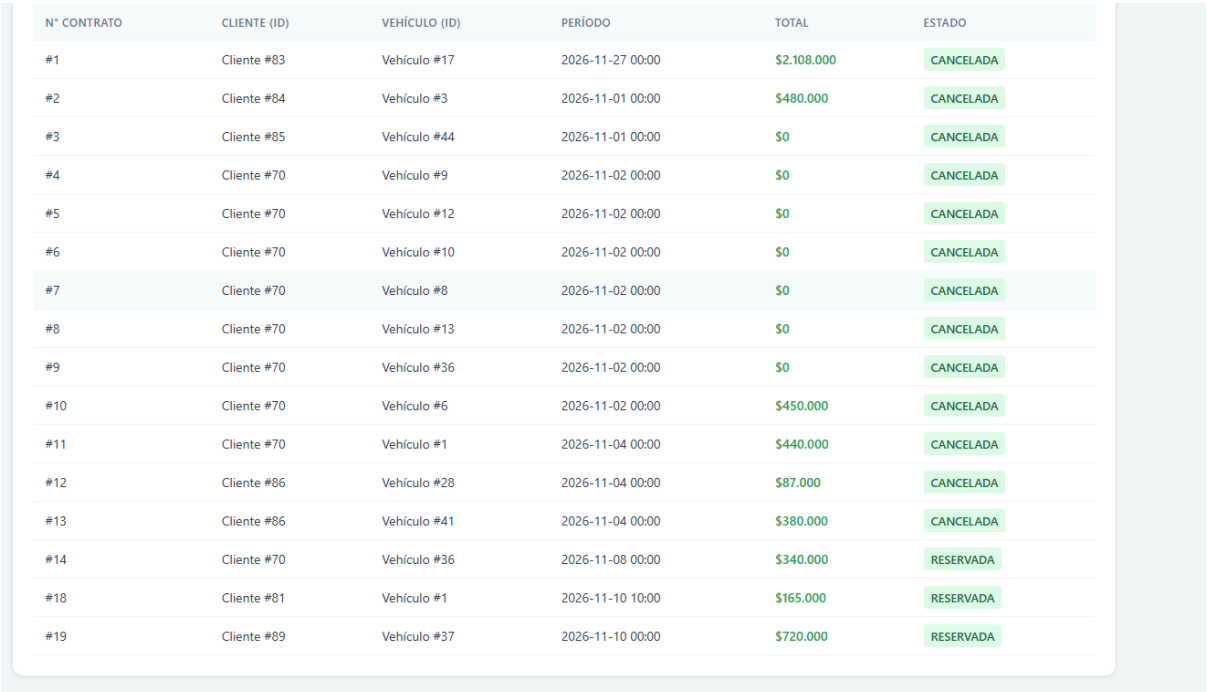
Para completar las funcionalidades del sistema a nivel de gestión gerencial y control operativo, desarrollé el panel de administración centralizado (`admin.html`), diseñado exclusivamente para los usuarios con privilegios de administrador. Este módulo permite supervisar el estado financiero, auditar los contratos de alquiler y dar de alta nuevas unidades en la flota de forma centralizada.

En la sección superior del panel implementé una grilla de métricas en tiempo real (`metrics-grid`) orientada a ofrecer un resumen ejecutivo de la operación del negocio:

Total Alquileres (`statAlquileres`): Muestra el volumen histórico y activo de contratos registrados en la base de datos (con un conteo operativo inicial de 16 contratos).

Flota Activa (`statVehiculos`): Monitorea la cantidad total de unidades operativas disponibles y en circulación dentro de las sucursales (registrando un parque activo de 191 unidades).

Ingresos Estimados (`statIngresos`): Calcula y proyecta de manera automática la recaudación estimada proveniente de los alquileres activos y finalizados (alcanzando cifras globales del orden de los $5.170.000).



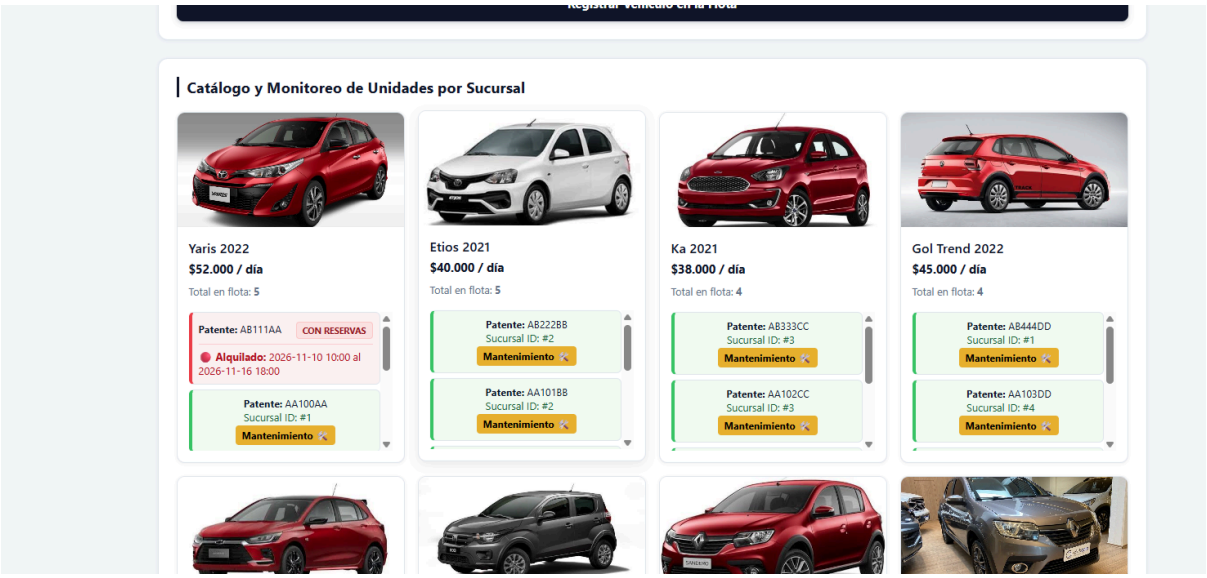
N° CONTRATO	CLIENTE (ID)	VEHÍCULO (ID)	PERÍODO	TOTAL	ESTADO
#1	Cliente #83	Vehículo #17	2026-11-27 00:00	$2.108.000	CANCELADA
#2	Cliente #84	Vehículo #3	2026-11-01 00:00	$480.000	CANCELADA
#3	Cliente #85	Vehículo #44	2026-11-01 00:00	$0	CANCELADA
#4	Cliente #70	Vehículo #9	2026-11-02 00:00	$0	CANCELADA
#5	Cliente #70	Vehículo #12	2026-11-02 00:00	$0	CANCELADA
#6	Cliente #70	Vehículo #10	2026-11-02 00:00	$0	CANCELADA
#7	Cliente #70	Vehículo #8	2026-11-02 00:00	$0	CANCELADA
#8	Cliente #70	Vehículo #13	2026-11-02 00:00	$0	CANCELADA
#9	Cliente #70	Vehículo #36	2026-11-02 00:00	$0	CANCELADA
#10	Cliente #70	Vehículo #6	2026-11-02 00:00	$450.000	CANCELADA
#11	Cliente #70	Vehículo #1	2026-11-04 00:00	$440.000	CANCELADA
#12	Cliente #86	Vehículo #28	2026-11-04 00:00	$87.000	CANCELADA
#13	Cliente #86	Vehículo #41	2026-11-04 00:00	$380.000	CANCELADA
#14	Cliente #70	Vehículo #36	2026-11-08 00:00	$340.000	RESERVADA
#18	Cliente #81	Vehículo #1	2026-11-10 10:00	$165.000	RESERVADA
#19	Cliente #89	Vehículo #37	2026-11-10 00:00	$720.000	RESERVADA

Debajo de las métricas, incorporé una tabla dinámica de control estructurada para auditar los contratos registrados (`tablaReservasBody`). Este componente se alimenta directamente de los Servlets y permite visualizar de forma tabular:

N° de Contrato y Clientes/Vehículos (IDs): Identificación unívoca de cada operación relacionada con las claves foráneas de la base de datos.

Períodos y Montos: Fechas de ejecución del servicio y costos asociados formateados con su respectiva moneda.

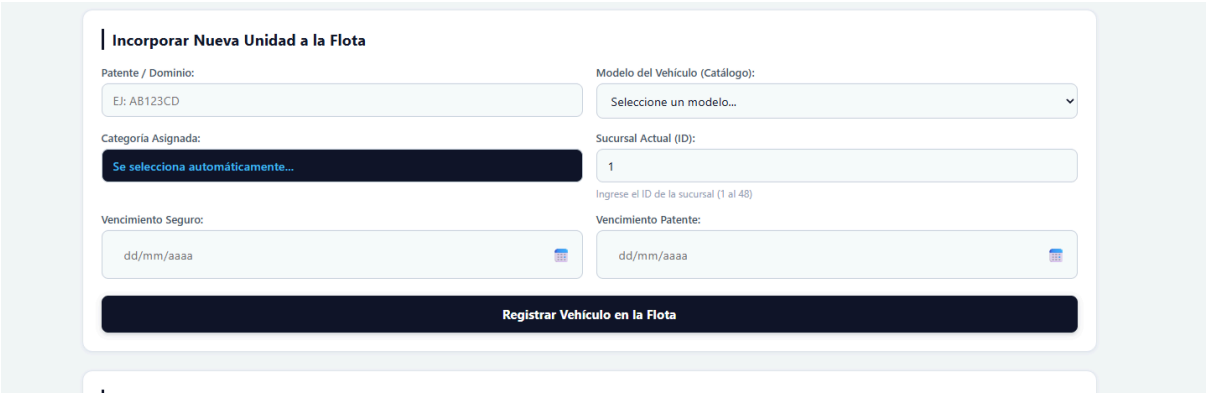
Estados de Contrato: Etiquetas visuales de control (como `CANCELADA` o `RESERVADA`) que permiten a los administradores conocer el estado operativo exacto de cada vehículo alquilado en tiempo real.



. Formulario de Incorporación de Nueva Unidad a la Flota

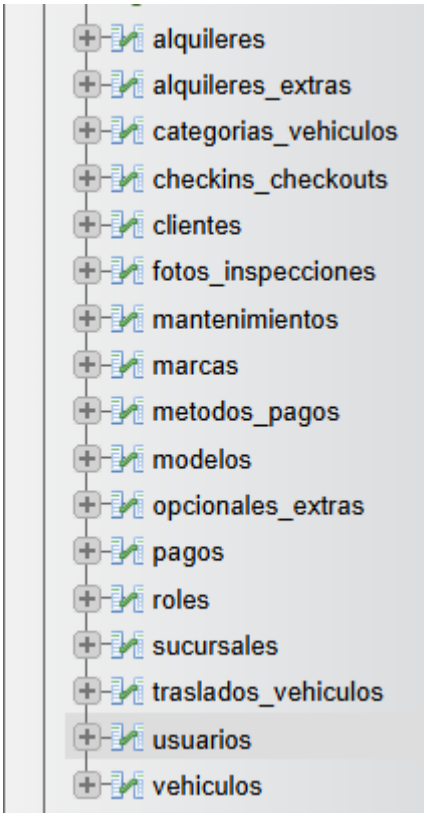
Para permitir que los administradores den de alta vehículos nuevos sin necesidad de manipular directamente la base de datos relacional, desarrollé un formulario interactivo específico (`formNuevoVehiculo`). Este módulo está estructurado mediante campos optimizados y selectores dinámicos:

- **Dominio y Patente:** Un campo de entrada con validación de longitud máxima (8 caracteres) y transformación automática a mayúsculas para cumplir con el formato legal de patentes (*Ej: AB123CD*).
- **Selector de Modelos del Catálogo (`nuevoModelo`):** Un menú desplegable conectado a los 40 modelos predefinidos en el sistema (desde vehículos generales como el Toyota Corolla 2023 o Volkswagen Amarok 2024 hasta unidades exclusivas de alta gama como la Ferrari 488 GTB o el Lamborghini Urus).
- **Asignación de Sucursal y Fechas:** Campos para ingresar el ID de la sucursal operativa (del 1 al 48) y selectores de fecha personalizados para el control de vencimientos de seguro y patente.



Implementación de la Capa de Acceso a Datos (DAOs) para las 17 Tablas del Sistema

Para garantizar una persistencia robusta y ordenada en la base de datos MySQL, el desarrollo de la arquitectura se estructuró implementando objetos de acceso a datos (**DAO** - *Data Access Object*) específicos para cada entidad del dominio. Dado que la base de datos cuenta con un modelo relacional complejo compuesto por **17 tablas interconectadas**, cada una requirió su propia interfaz y su clase de implementación concreta (`*DaoImpl`), operando de forma conjunta mediante el Patrón Factory.

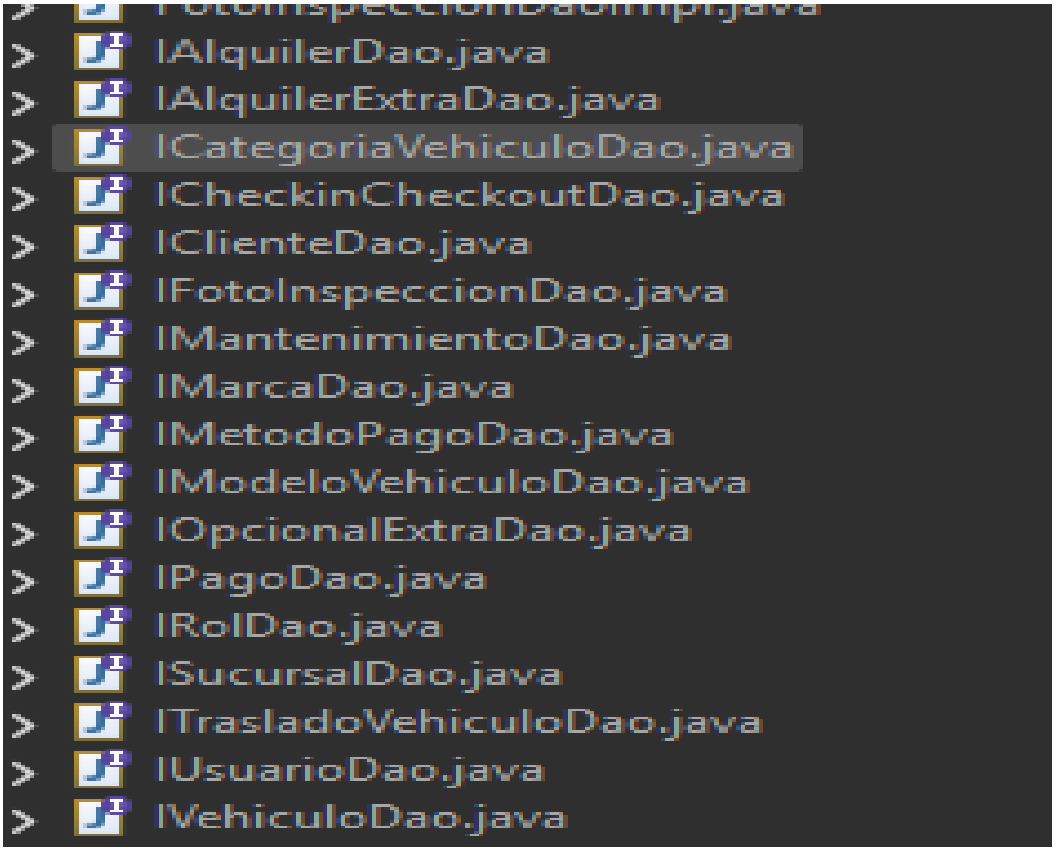


Estructuración y Patrón de Acceso a Datos

La separación de responsabilidades en esta capa se diseñó bajo los siguientes lineamientos técnicos:

Interfaces Genéricas por Entidad: Se definieron contratos estrictos para cada módulo del sistema (como `IMarcaDao`, `IUsuarioDao`, `IAlquilerDao`, `IVehiculoDao`, entre otros), asegurando que los métodos estándar de manipulación de datos (*CRUD: Create, Read, Update, Delete*) se mantengan uniformes en todo el proyecto.

Uso de JDBC y Conexión Centralizada: Las clases de implementación (`DaoImpl`) se conectan al motor relacional a través de `JDBC` utilizando una clase de gestión de conexiones centralizada que asegura la apertura y el cierre seguro de los recursos para evitar fugas de memoria en el servidor.



Desarrollo Metodológico de las Clases de Implementación DAO

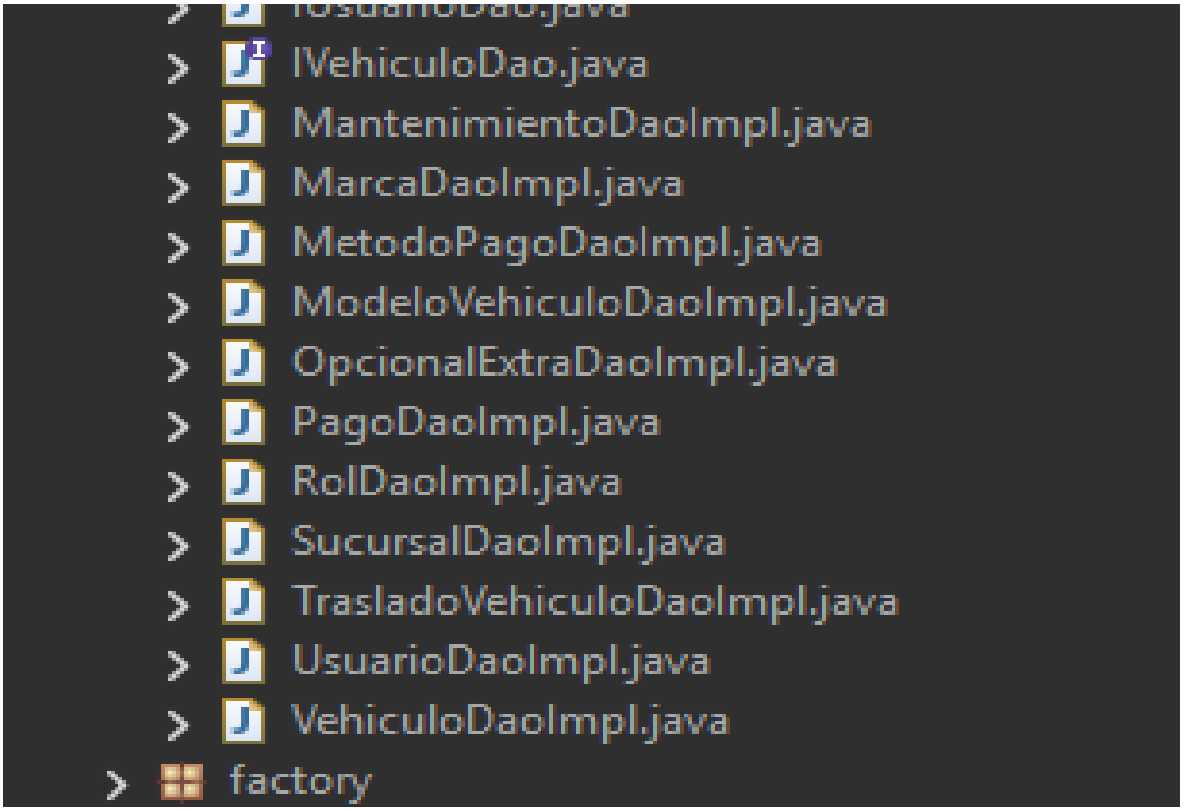
Para consolidar la capa de acceso a datos del sistema **"Los Pipiautos"** y dar cumplimiento a la arquitectura estipulada para las 17 tablas del modelo relacional, se desarrollaron las clases de implementación concreta (`*DaoImpl`). Estas clases encapsulan la lógica de comunicación con la base de datos MySQL, utilizando sentencias preparadas y llamadas seguras a procedimientos almacenados.

Cada clase de implementación (como se ejemplifica en el manejo de entidades asociadas y tablas intermedias) se diseñó bajo un patrón uniforme que garantiza la seguridad y el manejo transparente de excepciones:

> **Conexión Centralizada:** Todas las operaciones de inserción, modificación, eliminación y consulta abren y cierran los recursos de forma segura mediante bloques `try-with-resources` utilizando el gestor de conexiones del sistema (`config.Conexion.getInstancia().getConnection()`), previniendo fugas de memoria o conexiones huérfanas en el servidor.
>
> **Delegación mediante Stored Procedures:** Para evitar la exposición directa de consultas SQL en el código Java y optimizar el rendimiento del motor relacional, los métodos ejecutan llamadas a procedimientos almacenados utilizando `CallableStatement` (por ejemplo, `{CALL sp_insertar_alquiler_extra(?, ?, ?)}`), asignando los parámetros posicionales de manera estrictamente tipada (`setInt`, `setString`, etc.).

El procesamiento de las consultas de lectura (`SELECT`) se estructuró implementando métodos de mapeo independientes (`mapearAlquilerExtra(ResultSet rs)`), los cuales se encargan de transformar los registros tabulares provenientes de la base de datos en objetos limpios de la capa de modelo (`modelo.*`).

De esta manera, tanto en las consultas por identificador unívoco como en los listados generales (`listarTodos()`), se asegura que las claves primarias, foráneas y atributos particulares coincidan exactamente con la estructura normalizada de las 17 tablas, garantizando la estabilidad absoluta en la transferencia de datos hacia la lógica de negocio y los Servlets.



Modelado Dinámico y Diagramas de Secuencia del Sistema

Para documentar la interacción temporal y el intercambio de mensajes entre los distintos componentes de la arquitectura del software en **"Los Pipiautos"** (desde la interfaz de usuario hasta la persistencia en la base de datos), se desarrollaron **cuatro diagramas de secuencia** clave que reflejan los flujos operativos principales del negocio.

Diagrama de Secuencia 1: Autenticación y Validación de Usuario (`secuencias_1`)

Este primer diagrama modela el proceso de inicio de sesión analizado previamente en la interfaz web:

> **Interacción:** El actor (*Cliente* o *Administrador*) ingresa sus credenciales en el formulario del Front-End.
>
> **Flujo Asíncrono:** Mediante una petición `fetch`, el navegador envía los datos al Servlet correspondiente (`api/auth/login`).
>
> **Resolución y Persistencia:** El backend invoca al DAO de usuarios para validar los datos contra la base de datos MySQL, devolviendo un objeto JSON con los privilegios y generando el almacenamiento de la sesión en el navegador (`localStorage`).

2. Diagrama de Secuencia 2: Consulta y Renderizado de Flota y Catálogos (`secuencia_2.png`)

Modela el flujo de visualización de los vehículos en la página principal (`index.html`):

- **Interacción:** El cliente accede al catálogo general o de lujo.
- **Flujo Asíncrono:** El script del Front-End realiza una solicitud al Servlet de vehículos.
- **Procesamiento y Mapeo:** El controlador invoca al Factory y al DAO correspondiente, ejecutando el procedimiento almacenado en MySQL para recuperar los registros y serializarlos a formato JSON mediante la librería **Gson**, permitiendo que JavaScript dibuje dinámicamente las tarjetas (*cards*) en pantalla.

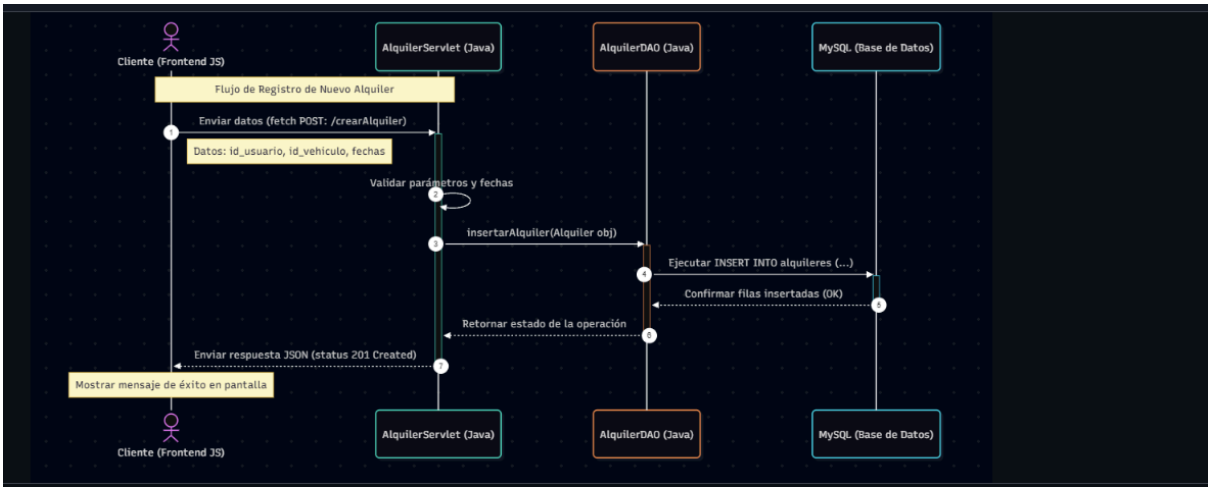


Diagrama de Secuencia 3: Gestión y Registro de Nuevos Alquileres (`secuencia_3.png`)

Detalla la lógica transaccional de una reserva:

- **Interacción:** El usuario selecciona un vehículo, indica el período y confirma el alquiler.
- **Validación de Negocio:** El Servlet y las capas de servicio verifican la disponibilidad de fechas para evitar superposiciones de reservas sobre una misma unidad.
- **Impacto en Base de Datos:** Se ejecuta el alta en las tablas relacionales y entidades intermedias (como el manejo de extras y estados de contrato), actualizando el stock disponible en tiempo real para las sucursales.

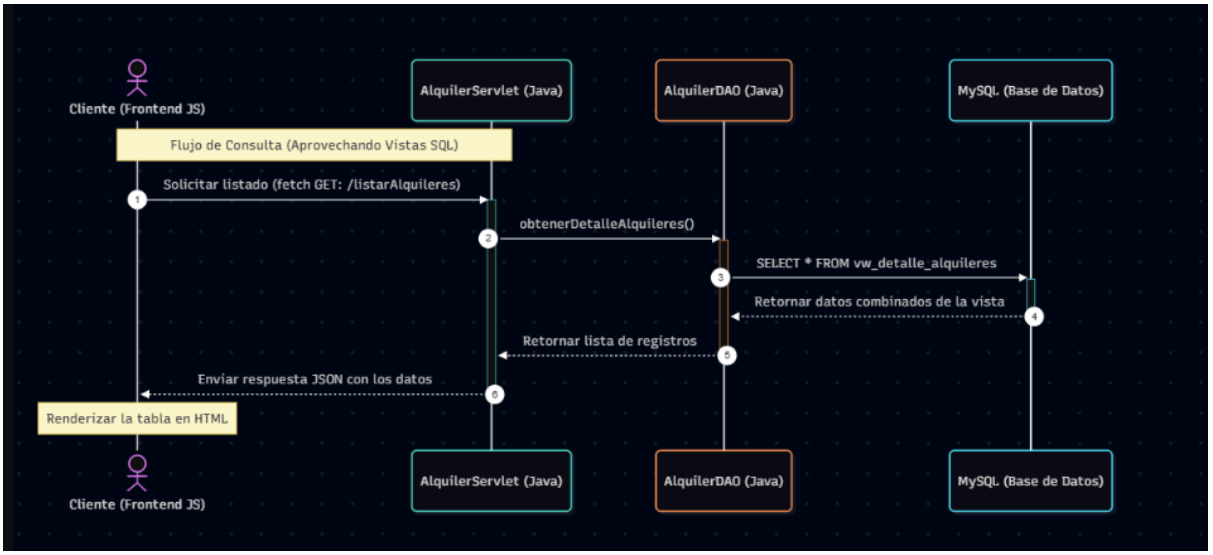
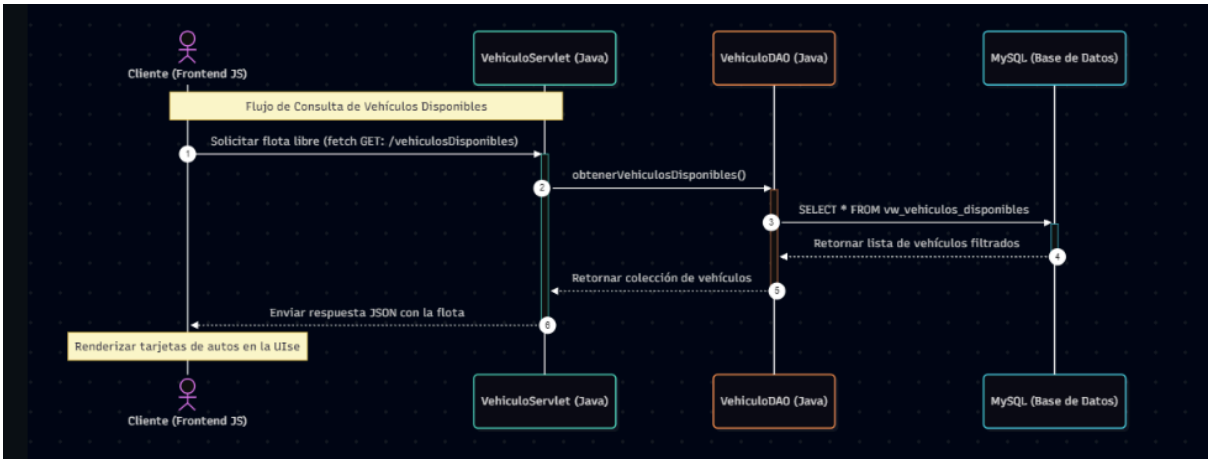


Diagrama de Secuencia 4: Panel de Administración y Monitoreo de Unidades (`secuencia_4.png`)

Representa el flujo de control gerencial ejecutado desde el `admin.html`:

- **Interacción:** El administrador ingresa al dashboard y visualiza las métricas de alquileres, ingresos y flota activa.
- **Alta de Unidades:** A través del formulario de incorporación, se envían los datos de patente, modelo (asociado al catálogo de 40 opciones) y sucursal.
- **Persistencia y Respuesta:** El sistema procesa la inserción asegurando la integridad transaccional y actualiza el catálogo visual de monitoreo por sucursal.



Conclusión y Evaluación Final del Proyecto

El desarrollo e implementación integral de la plataforma de alquiler de vehículos **"Los Pipiautos"** permitió consolidar y aplicar de forma práctica los conocimientos teóricos y técnicos adquiridos a lo largo de la especialidad en computación. A lo largo de este proyecto, se logró diseñar y poner en marcha un sistema informático funcional, escalable y robusto, abarcando desde el modelado de bases de datos relacionales hasta la maquetación de interfaces web modernas y el desarrollo de la lógica de backend.

Entre los logros técnicos más destacados del proyecto se encuentran:

Persistencia Robusta: La estructuración de un modelo relacional avanzado en MySQL, operando de manera eficiente mediante la implementación de patrones DAO, sentencias preparadas y procedimientos almacenados (`CallableStatement`) que garantizan la integridad y seguridad de los datos.

Arquitectura de Software Desacoplada: La correcta separación de responsabilidades entre el cliente (HTML5, CSS3 y JavaScript con peticiones asíncronas `fetch`) y el servidor (Servlets, Java y objetos de modelo), permitiendo una comunicación fluida y en tiempo real.

Experiencia de Usuario y Control de Negocio: La creación de una interfaz responsiva y atractiva, dividida entre un catálogo general por marcas, una sección exclusiva de alta gama con restricciones de acceso, un sistema de gestión por sucursales y un Panel de Administración (`admin.html`) totalmente operativo para el monitoreo de flotas y auditoría de contratos.

Limitaciones de Alcance y Perspectiva Operativa

No obstante el alcance alcanzado, es importante señalar que durante el proceso de desarrollo y debido a los plazos estipulados para la entrega, algunas entidades secundarias proyectadas en el modelo teórico inicial tales como tablas de auditoría avanzada de sesiones o módulos detallados de facturación impositiva quedaron temporalmente fuera de la versión final de producción. Asimismo, se prescindió deliberadamente de tablas complejas como las de **inspección fotográfica de daños o control detallado de taller y mantenimiento de piezas**, dado que el sistema opera bajo una lógica de gestión comercial de flotas y alquileres, y no cuenta con un espacio físico de taller mecánico ni infraestructura de postventa asociada en esta etapa. Esta priorización consciente permitió concentrar los recursos de programación en consolidar los módulos troncales del sistema, asegurando que las tablas operativas y los flujos principales funcionen con absoluta estabilidad.

Final

Mirando el recorrido completo de **"Los Pipiautos"**, este proyecto dejó de ser un simple requerimiento técnico para convertirse en la materialización de un sistema de software real, diseñado y resuelto de punta a punta. Desde el modelado inicial de una base de datos compleja con 17 tablas y su integración limpia mediante DAOs en Java, hasta la construcción de una interfaz web moderna, responsiva y conectada mediante peticiones asíncronas, cada módulo reflejó el dominio de la lógica de programación y la arquitectura de sistemas.

En el camino quedó demostrado que desarrollar software no es solo escribir código, sino tomar decisiones estratégicas: saber priorizar los pilares fundamentales del sistema como el catálogo dinámico, la gestión de la red federal, el control de usuarios y el panel de administración operativo y descartar con criterio aquello que excedía el alcance operativo real. El resultado es una plataforma robusta, funcional y coherente, que consolida con solidez el perfil técnico y demuestra la capacidad de llevar una solución informática desde la arquitectura en papel hasta su despliegue integral.